

Luiss

Libera Università Internazionale
degli Studi Sociali Guido Carli

Dictionaries, Tuples, and Sets

Introduction to Computer Programming

Management and Artificial Intelligence

LUISS



Collections

In Python, **collections** are container (non-scalar) data types:

- Collections allow you to store **multiple values** in a single data structure
- They provide **one variable** that contains many elements
- Each element has a specific **position or key** within the collection
- Collections include **methods to access** individual elements efficiently

Python has four built-in collection data types:

- lists (covered in previous lecture)
- dictionaries
- tuples
- sets

A Story of Two Collections

Collection	Features	Visualization
List	a linear ordered collection of values list index their entities based on the position in the list.	
Dictionary	a " bag of values ", each with its own label the order of items in a dictionary is unpredictable dictionaries are indexed by keys (lookup tags)	



Dictionaries

- it maps a given <key> to a given <value>, hence, it stores *key-value pairs*
- <key> can be any **immutable** value (actually, it must be a **hashable** value)
 - hashable data types are: int, float, str, tuple, bool, None
- <value> can be anything (mutable, immutable, heterogenous)
- In other languages, dictionaries are also known as *hash tables* or *associative arrays*
- the (<key, <value>) pairs are sorted without a specific ordering (it was implementation specific, while in the last releases it is ordered but you should not rely on it)
- it is *mutable*, i.e., we can update its content (e.g., add/remove elements)
- it is denoted by curly brackets ({ at the begin and } at the end), with pairs separated by a comma , , where each pair has : to separate the key from the value

For instance:

```
d = {1: 'one', '2': 2.0, 3.0: 3} # dictionary with three key-value pairs
print(d)
```

```
{1: 'one', '2': 'two', 3.0: 'three'}
```

Creating a Dictionary

There are two ways for creating a dictionary:

Creating a Dictionary

There are two ways for creating a dictionary:

1. Step-by-step

```
d1 = dict()           # or d1 = {}, i.e., empty dictionary
d1['money'] = 12       # add a new key-value pair
d1['candy'] = 3        # add a new key-value pair
d1['tissues'] = 75    # add a new key-value pair
print(d1)
```

```
{'money': 12, 'candy': 3, 'tissues': 75}
```

Creating a Dictionary

There are two ways for creating a dictionary:

1. Step-by-step

```
d1 = dict()           # or d1 = {}, i.e., empty dictionary
d1['money'] = 12       # add a new key-value pair
d1['candy'] = 3        # add a new key-value pair
d1['tissues'] = 75    # add a new key-value pair
print(d1)
```

```
{'money': 12, 'candy': 3, 'tissues': 75}
```

2. All-at-once

```
d2 = {'money': 12, 'candy': 3, 'tissues': 75}
print(d2)
```

```
{'money': 12, 'candy': 3, 'tissues': 75}
```

Indexing and Manipulating a Dictionary

Indexing and Manipulating a Dictionary

- **Dictionary Query**

```
d = {'money': 12, 'candy': 3, 'tissues': 75}  
print(d['money'])
```

12

Indexing and Manipulating a Dictionary

- **Dictionary Query**

```
d = {'money': 12, 'candy': 3, 'tissues': 75}  
print(d['money'])
```

12

- **Update the Dictionary**

```
d['candy'] = d['candy'] + 1  
print(d)
```

{'money': 12, 'candy': 4, 'tissues': 75}

- **Add New Items to the Dictionary**

```
d['pen'] = 2  
print(d)
```

```
{'money': 12, 'candy': 4, 'tissues': 75, 'pen': 2}
```

- **Add New Items to the Dictionary**

```
d['pen'] = 2  
print(d)
```

```
{'money': 12, 'candy': 4, 'tissues': 75, 'pen': 2}
```

- **Delete Items from the Dictionary**

```
del(d['money']) # del works on lists too...  
print(d)
```

```
{'candy': 4, 'tissues': 75, 'pen': 2}
```

WARNING:

If the index is not a key in the dictionary, Python raises a dedicated **KeyError** error.

For instance:

```
d = {'money': 12, 'candy': 3, 'tissues': 75}
print(d['hello'])
```

KeyError

Traceback (most recent call last)

Cell In[7], line 2

```
1 d = {'money': 12, 'candy': 3, 'tissues': 75}
----> 2 print(d['hello'])
```

KeyError: 'hello'

Operator `in`

The operator `in` can be used to check if a **key** is present in a collection:

Operator `in`

The operator `in` can be used to check if a **key** is present in a collection:

```
d = {"a": 1, "b": 2, "c": 3}

if "a" in d:
    print("a is in d")
else:
    print("a is not in d")
```

a is in d

Operator `in`

The operator `in` can be used to check if a **key** is present in a collection:

```
d = {"a": 1, "b": 2, "c": 3}

if "a" in d:
    print("a is in d")
else:
    print("a is not in d")
```

a is in d

```
if 1 in d:
    print("1 is in d with the value", d[1])
else:
    print("1 is not in d")
```

1 is not in d

WARNING:

The `in` operator behaves differently with respect to lists and strings:

- For offset-indexed sequences (e.g., strings and lists), `x in y` checks whether `x` is an item in the sequence `y`
- For dictionaries, `x in y` checks whether `x` *is a key* of the dictionary `y`

```
L = [4, 6, 'hello', 7] # a list
print(7 in L)
s = 'coding with python' # a string
print('p' in s)
d = {'money': 12, 'candy': 3, 'tissues': 75} # a dict
print('hello' in d)
print('money' in d)
```

```
True
True
False
True
```

Dict Methods

dict.keys()

`dict.keys()` returns a *view object** containing all the keys of a dictionary. You can conveniently cast into a list, a more common and flexible data object.

For instance:

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
k = dishes.keys() # a view object (list-like)
print(k)
k = list(k) # a proper list
print(k)
```

```
dict_keys(['eggs', 'sausage', 'bacon', 'spam'])
['eggs', 'sausage', 'bacon', 'spam']
```

WARNING: Dictionaries keep their items in the same order they were introduced**: keys, values and items (more on the latter later) are ordered according to this rule.

* View objects are dynamic overview on the dictionary's entries

** In Python 3.6 and earlier, dictionaries are fully unordered.

`dict.items()` and `dict.values()`

`dict.values()` returns a *view object* containing all the values of a dictionary.

`dict.items()` returns a *view object* containing all the key-value pairs of a dictionary.

You can conveniently cast into a list, a more common and flexible data object.

For instance:

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
v = dishes.values() # a view object (list-like)
v = list(v) # a proper list
print(v)
i = dishes.items() # a view object (list-like)
i = list(i) # a list of key-value tuples
print(i)
```

```
[2, 1, 1, 500]
```

```
[('eggs', 2), ('sausage', 1), ('bacon', 1), ('spam', 500)]
```

Hence, to check whether a value is within a key-value pair within a dictionary, we can use the `in` operator and the `values()` method:

```
d = {"a": 1, "b": 2, "c": 3}
if 1 in d.values():
    print("Value 1 is in the dictionary")
else:
    print("Value 1 is not in the dictionary")
```

Value 1 is in the dictionary

```
if "a" in d.values():
    print("Value a is in the dictionary")
else:
    print("Value a is not in the dictionary")
```

Value a is not in the dictionary

dict.get()

`dict.get(key, [x])` returns the value for `key`. If `key` is not found, returns `x`. If `x` is not given, it defaults to `None`.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
v = dishes.get('eggs', 10) # 'eggs' exists in dishes
print(v)
v = dishes.get('water', 10) # 'water' does not exist in dishes
print(v)
```

```
2
10
```

NOTE: Since `dict.get()` relies on a default value if key is not found, it will never raise a `KeyError`.

`dict.get()` explained:

Let `d` be a dictionary. Saying `v = d.get(k, x)` is a shortcut for:

```
if k in d:  
    v = d[k]  
else:  
    v = x
```

Similarly, saying `d[k] = d.get(k, x) + y` is a shortcut for:

```
if k in d:  
    d[k] += y  
else:  
    d[k] = x + y
```

PATTERN OF USAGE: check whether a key is already in a dictionary and assuming a default value if the key is not there.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
# 'eggs' exists in dishes, so we take its value and add + 1
dishes['eggs'] = dishes.get('eggs', 0) + 1
print(dishes)
# 'water' does not exists in dishes, so we plug the default value 0 and add + 1
dishes['water'] = dishes.get('water', 0) + 1
print(dishes)
```

```
{'eggs': 3, 'sausage': 1, 'bacon': 1, 'spam': 500}
{'eggs': 3, 'sausage': 1, 'bacon': 1, 'spam': 500, 'water': 1}
```


`dict.clear()`

`dict.clear()` deletes all items from a dictionary.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}  
dishes.clear()  
print(dishes)
```

```
{}
```

NOTE: There is a strong analogy with lists: `clear()` also works on lists (same effect → empty list).

dict.pop()

dict.pop(key) deletes the dictionary entry corresponding to the key key and returns its value.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
v = dishes.pop('sausage')
print(dishes)
print(v)           # value of the removed element
```

```
{'eggs': 2, 'bacon': 1, 'spam': 500}
1
```

NOTE: There is a strong analogy with lists: pop() also returns the value when called on a list.

dict.update()

`dict.update(other_dict)` updates the dictionary `dict` with the key-value pairs from `other_dict`, overwriting existing keys.

```
d1 = {1: 10, 2: 20}
d2 = {3: 30, 4: 40}
d1.update(d2)
print(d1)
```

```
{1: 10, 2: 20, 3: 30, 4: 40}
```

WARNING: If a key exists both `dict` and `other_dict`, the key-value pair in `other_dict` overwrites the one in `dict`.

```
d1 = {1: 10, 2: 20}
d2 = {3: 30, 2: 40} # we have an overlapping key (i.e., 2)
d1.update(d2)
print(d1) # the value associated to key 2 is taken from d2
```

```
{1: 10, 2: 40, 3: 30}
```

len of dictionary

The `len` function returns the number of key-value pairs in the dictionary:

```
d = {"a": 1, "b": 2, "c": 3}
print("The dictionary has {} key-value pairs.".format(len(d)))
```

The dictionary has 3 key-value pairs.

Iterating over Dicts

You can iterate over a dictionary with a very simple for loop:

- over its keys (way common)
- over its values (less common)

Iterating over Dicts

You can iterate over a dictionary with a very simple for loop:

- over its keys (way common)
- over its values (less common)

HINT: You can generate the sequence of keys and values with `dict.keys()` and `dict.values()`, respectively.

Iterating over Dicts

You can iterate over a dictionary with a very simple for loop:

- over its keys (way common)
- over its values (less common)

HINT: You can generate the sequence of keys and values with `dict.keys()` and `dict.values()`, respectively.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for k in dishes.keys():
    print("Key is:", k, "and Value is:", dishes[k])
```

```
Key is: eggs and Value is: 2
Key is: sausage and Value is: 1
Key is: bacon and Value is: 1
Key is: spam and Value is: 500
```

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for v in dishes.values():
    print("Value is:", v)
```

```
Value is: 2
Value is: 1
Value is: 1
Value is: 500
```



```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for v in dishes.values():
    print("Value is:", v)
```

```
Value is: 2
Value is: 1
Value is: 1
Value is: 500
```

HINT: If you iterate over keys, it's trivial to have both key and value. If you iterate over values, it's not trivial to get the corresponding key.

You can iterate **simultaneously** over keys and values of a dictionary with a very simple for loop.

You can iterate **simultaneously** over keys and values of a dictionary with a very simple for loop.

HINT: You can get the key-value pairs with `dict.items()`.

You can iterate **simultaneously** over keys and values of a dictionary with a very simple for loop.

HINT: You can get the key-value pairs with `dict.items()`.

```
dishes = {'eggs': 2, 'sausage': 1, 'bacon': 1, 'spam': 500}
for k, v in dishes.items():
    print("Key is:", k, "and Value is:", v)
```

```
Key is: eggs and Value is: 2
Key is: sausage and Value is: 1
Key is: bacon and Value is: 1
Key is: spam and Value is: 500
```

What's going on here?

- there are **two iteration variables**, namely `k` and `v`
- `dict.items()` generates a list of key-value pairs (tuple)
- the first loop variable, namely `k`, contains the key in the key-value pair
- the second loop variable, namely `v`, contains the corresponding value in the key-value pair

Tuples

Tuples:

- like lists, are **ordered collections** of items.
- unlike lists, are **immutable**:
 - after creation, you cannot add or remove items.
 - if the items are mutable, they can still be updated.
- defined using **parentheses** instead of square brackets.

Creating Tuples

You can create a tuple by enclosing items in round brackets:

```
t1 = (90, 98, 95) # homogeneous tuple  
t2 = (90, 'hello', 95, [1, 2]) # heterogeneous tuple  
t3 = () # empty tuple
```

Creating Tuples

You can create a tuple by enclosing items in round brackets:

```
t1 = (90, 98, 95) # homogeneous tuple
t2 = (90, 'hello', 95, [1, 2]) # heterogeneous tuple
t3 = () # empty tuple
```

or, you can omit the parenthesis altogether (this is called *tuple packing*):

```
t1 = 90, 98, 95 # homogeneous tuple
t2 = 90, 'hello', 95, [1, 2] # heterogeneous tuple
```

Creating Tuples

You can create a tuple by enclosing items in round brackets:

```
t1 = (90, 98, 95) # homogeneous tuple
t2 = (90, 'hello', 95, [1, 2]) # heterogeneous tuple
t3 = () # empty tuple
```

or, you can omit the parenthesis altogether (this is called *tuple packing*):

```
t1 = 90, 98, 95 # homogeneous tuple
t2 = 90, 'hello', 95, [1, 2] # heterogeneous tuple
```

WARNING: When you create a singleton tuple, you must include a trailing comma!

```
t = (90) # type(t) yields 'int'
print(type(t))
t = (90,) # now we have a 1-element tuple
print(type(t))
```

```
<class 'int'>
<class 'tuple'>
```


Unpacking

Tuples (and lists) can also be *unpacked**:

```
t = (90, 0, 'hello')  
a, b, c = t # unpacking syntax  
print(a, b, c)
```

90 0 hello

which is basically the same as doing:

```
a = t[0]  
b = t[1]  
c = t[2]  
print(a, b, c)
```

90 0 hello

*: Unpacking is when you assign values from a tuple/list to a sequence of variables.

Common operations

Tuples support all of the common operations we have for lists, notably:

Common operations

Tuples support all of the common operations we have for lists, notably:

- **indexing:**

```
t = (98, 7, 'hello')  
t[1]
```

7

Common operations

Tuples support all of the common operations we have for lists, notably:

- **indexing:**

```
t = (98, 7, 'hello')  
t[1]
```

7

- **slicing:**

```
t[1:3]
```

(7, 'hello')

Common operations

Tuples support all of the common operations we have for lists, notably:

- **indexing:**

```
t = (98, 7, 'hello')  
t[1]
```

7

- **slicing:**

```
t[1:3]
```

(7, 'hello')

- the + operator for **concatenation**:

```
t2 = (1, 2, 3)  
t3 = t + t2  
print(t3)
```

(98, 7, 'hello', 1, 2, 3)

- the * operator for **replication**:

```
t * 3
```

```
(98, 7, 'hello', 98, 7, 'hello', 98, 7, 'hello')
```

- the * operator for **replication**:

```
t * 3
```

```
(98, 7, 'hello', 98, 7, 'hello', 98, 7, 'hello')
```

- the in (and not in) operator for **membership checking**:

```
if 7 in t:  
    print("7 is in t")  
else:  
    print("7 is not in t")
```

```
7 is in t
```

```
if 8 not in t:  
    print("8 is not in t")  
else:  
    print("8 is in t")
```

```
8 is not in t
```

- the `len()` function to calculate the **length of a tuple**:

```
len(t)
```

3

Iterating over Tuples

You can also iterate over tuples, e.g. via for loops:

```
t = (98, 7, 'hello')  
for item in t:  
    print(item)
```

```
98  
7  
hello
```

Iterating over Tuples

You can also iterate over tuples, e.g. via for loops:

```
t = (98, 7, 'hello')
for item in t:
    print(item)
```

```
98
7
hello
```

Or, via a while loop by index:

```
t = (98, 7, 'hello')
i = 0
while i < len(t):
    print(t[i])
    i += 1
```

```
98
7
hello
```

Immutability

Yet, **tuples are immutable**, whereas **lists are mutable**.

```
t = (98, 7, 'hello')  
t[-1] = 'python'
```

TypeError

Traceback (most recent call last)

Cell In[25], line 2

```
1 t = (98, 7, 'hello')  
----> 2 t[-1] = 'python'
```

TypeError: 'tuple' object does not support item assignment

So you cannot change a tuple, just like you cannot change a string.

Workaround tuple immutability

Workarounds:

- create a new tuple that contains the manipulation of the original tuple
- cast tuple to list, change the list, cast list to tuple

As the latter is concerned:

```
t = (98, 7, 'hello')    # t is a tuple
print(t)
L = list(t) # convert tuple to list
print(L)
L.append(1) # do some manipulations
print(L)
t = tuple(L) # convert list to tuple
print(t)
```

```
(98, 7, 'hello')
[98, 7, 'hello']
[98, 7, 'hello', 1]
(98, 7, 'hello', 1)
```

Sets

A set is an **unordered** (unindexed) collection of items with **no** duplicates.

Creating sets

You can create an empty set thanks to the `set()` function.

And you can create a non-empty set thanks to the `set()` function or curly brackets.

For example:

```
# empty set  
s1 = set()  
# non-empty set with curly brackets  
s2 = {'a', 1}  
# non-empty set with set()  
s3 = set(['a', 1])
```

Creating sets

You can create an empty set thanks to the `set()` function.

And you can create a non-empty set thanks to the `set()` function or curly brackets.

For example:

```
# empty set  
s1 = set()  
# non-empty set with curly brackets  
s2 = {'a', 1}  
# non-empty set with set()  
s3 = set(['a', 1])
```

WARNING: To not confuse the curly brackets to create dicts with the curly brackets to create sets.

Data types storable in a set

A set can contain only ***hashable*** items, notably:

- numbers (`int`, `float`)
- strings (`str`)
- tuples only if they do not contain mutable objects

and in no way they can contain mutable objects, notably:

- dictionaries
- lists
- tuples that contain mutable objects

In Python, the majority of immutable objects are also hashable.

Hashable

Hashing is a technique in computer science used to create efficient data structures that allow for fast storage and retrieval of data.

Key properties of hashing:

- A hash function converts data into a fixed-size value (hash)
- The same input will always produce the same hash (deterministic)
- Mutable objects cannot be hashed because their content can change, which would change their hash value
- Comparing hashes is typically faster than comparing the original data, thanks to the fixed-size nature of hashes

In Python, sets and dictionaries rely on hashing for efficient lookup operations. This is why only hashable objects (like strings, numbers, and tuples of hashable objects) can be used as dictionary keys or set elements.

EXAMPLE: cost of membership check (`if x in X`)

- If X is a list, then Python scans the entire list until x is found: $O(n)$ time complexity (linear time, where n is the length of the list), meaning the operation takes longer as the list grows larger:

EXAMPLE: cost of membership check (`if x in X`)

- If `X` is a list, then Python scans the entire list until `x` is found: $O(n)$ time complexity (linear time, where n is the length of the list), meaning the operation takes longer as the list grows larger:

```
import time
L = [1] * 1000 + [2]
start = time.time()
if '2' in L: print('2 is in L')
elapsed_time_1 = time.time() - start
print(elapsed_time_1, "seconds")
```

3.743171691894531e-05 seconds

EXAMPLE: cost of membership check (`if x in X`)

- If `X` is a list, then Python scans the entire list until `x` is found: $O(n)$ time complexity (linear time, where n is the length of the list), meaning the operation takes longer as the list grows larger:

```
import time
L = [1] * 1000 + [2]
start = time.time()
if '2' in L: print('2 is in L')
elapsed_time_1 = time.time() - start
print(elapsed_time_1, "seconds")
```

3.743171691894531e-05 seconds

```
L = [1] * 1000000 + [2]
start = time.time()
if '2' in L: print('2 is in L')
elapsed_time_2 = time.time() - start
print(elapsed_time_2, "seconds [", elapsed_time_2 / elapsed_time_1, "times slower", '']
```

0.007437467575073242 seconds [198.69426751592357 times slower]

EXAMPLE: cost of membership check (`if x in X`)

- If `X` is a list, then Python scans the entire list until `x` is found: $O(n)$ time complexity (linear time, where n is the length of the list), meaning the operation takes longer as the list grows larger:

```
import time
L = [1] * 1000 + [2]
start = time.time()
if '2' in L: print('2 is in L')
elapsed_time_1 = time.time() - start
print(elapsed_time_1, "seconds")
```

3.743171691894531e-05 seconds

```
L = [1] * 1000000 + [2]
start = time.time()
if '2' in L: print('2 is in L')
elapsed_time_2 = time.time() - start
print(elapsed_time_2, "seconds [", elapsed_time_2 / elapsed_time_1, "times slower", "
```

0.007437467575073242 seconds [198.69426751592357 times slower]

We had a significant increase in the search time. With more complex values and more elements, the search time increases linearly. Try experimenting with larger datasets to observe this behavior!

- If X is a dictionary or set, $\text{hash}(x)$ gives the position in memory: $O(1)$ time complexity (constant time), meaning the operation takes the same amount of time regardless of the collection's size:

- If X is a dictionary or set, `hash(x)` gives the position in memory: $O(1)$ time complexity (constant time), meaning the operation takes the same amount of time regardless of the collection's size:

```
import time
D = {str(i): i for i in range(1000, 0, -1)}
start = time.time()
if '2' in D: print('2 is in D')
elapsed_time_1 = time.time() - start
print(elapsed_time_1, "seconds")
```

```
2 is in D
0.0014166831970214844 seconds
```

- If X is a dictionary or set, `hash(x)` gives the position in memory: $O(1)$ time complexity (constant time), meaning the operation takes the same amount of time regardless of the collection's size:

```
import time
D = {str(i): i for i in range(1000, 0, -1)}
start = time.time()
if '2' in D: print('2 is in D')
elapsed_time_1 = time.time() - start
print(elapsed_time_1, "seconds")
```

```
2 is in D
0.0014166831970214844 seconds
```

```
import time
D = {str(i): i for i in range(1000000, 0, -1)}
start = time.time()
if '2' in D: print('2 is in D')
elapsed_time_2 = time.time() - start
print(elapsed_time_2, "seconds [", elapsed_time_2 / elapsed_time_1, "times slower", ']
```

```
2 is in D
0.00044465065002441406 seconds [ 0.31386738471894987 times slower ]
```


NOTE: You will come back to concept of time complexity next year during the Algorithms course.

Common operations

- **membership**: $x \in S$
- the `len(s)` function to calculate the **cardinality** (size) of a set `s`
- **union** (new set with elements in either `s1` or `s2`): $s1 \cup s2$ or `s1.union(s2)`
- **intersection** (new set with elements in both `s1` and `s2`): $s1 \cap s2$ or `s1.intersection(s2)`
- **difference** (new set with elements in `s1` but not in `s2`): $s1 - s2$ or `s1.difference(s2)`
- **symmetric difference** (new set with elements in `s1` or `s2`, but not both): $s1 \oplus s2$ or `s1.symmetric_difference(s2)`

NOTE: Set operations yield a brand new set.

Set methods

- `s.add(x)`: adds `x` to the set `s` (if `x` already in `s`, nothing happens)
- `s.remove(x)`: removes `x` from the set `s` (if `x` is not in `s`, a `KeyError` is raised)
- `s.discard(x)`: removes `x` from the set `s` (if `x` is not in `s`, nothing happens)
- `s.update(x)`: adds elements from `x` into the set `s` (`x` can be any iterable, e.g., list, tuple or another set)
- `s.clear()`: removes all elements from `s`
- `s.pop()`: removes (and returns) an arbitrary item from `s`

NOTE: Set methods change the set in-place.

Recap on Collections

- `list` and `tuple` keep order; `set` and `dict` do not
- `dict` associates a key with a value; `set`, `tuple` and `list` contain just values
- `set` and `dict` keys only require items to be hashable; `list`, `tuples` and `dict` values do not
- `set` and `dict` keys forbid duplicates; `list`, `tuples` and `dict` values allow duplicates.

Checking for membership inside a `set` or `dict` is super-fast thanks to hashing.

`tuple` and `list` require scanning the list/tuple until the element is found.

